Tileserver Polars: A Hybrid Python–Rust Vector Tile Engine

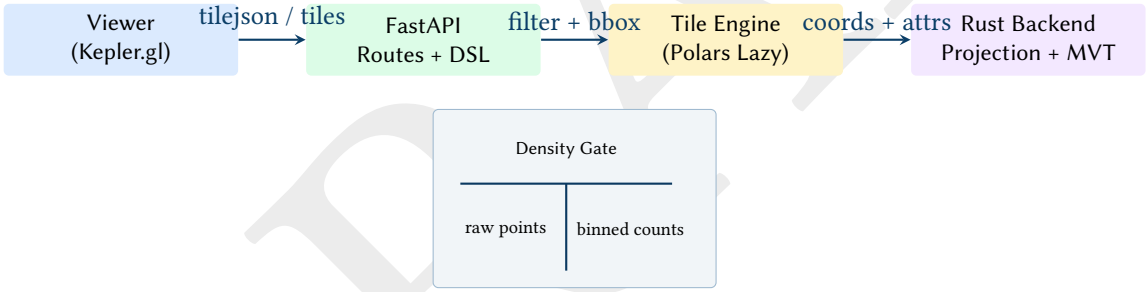
Columnar Filtering, Tile-Space Projection, and MVT Encoding for Interactive Point Clouds

Simon Knight

Adelaide, Australia

March 2026 • Version 0.1.0

Last updated: March 12, 2026



Abstract. This report documents a high-performance vector tile server for large point datasets backed by Parquet and Polars. The system uses Polars lazy expressions for server-side predicate pushdown and a Rust extension for Web Mercator tile-space projection and Mapbox Vector Tile (MVT) encoding. A density-gated mode switch emits either raw points (attribute-rich) or adaptive grid bins (count-only) to maintain interactive responsiveness at high point counts. We describe the API surface, the filter DSL, the tile engine, and the Rust MVT encoder, and provide implementation diagrams in TikZ for traceable system intent.

Contents

List of Design Decisions & Rules

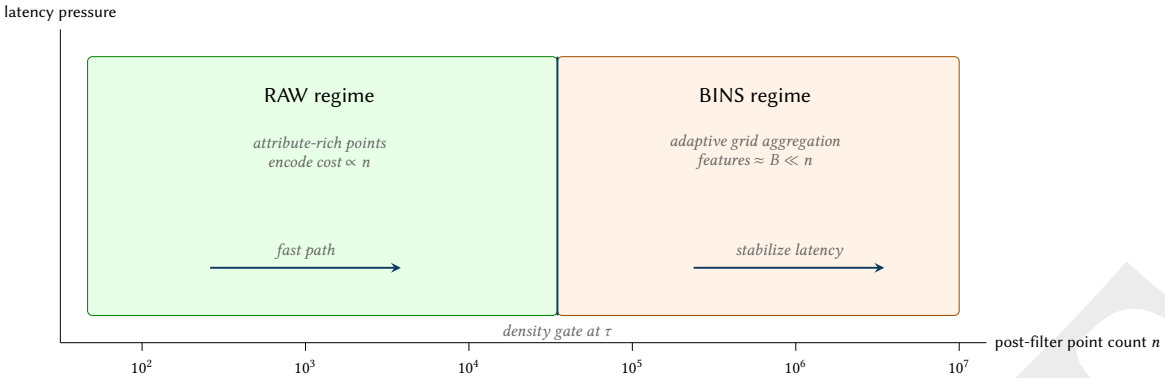


Figure 1. Performance regime diagram: the tile engine switches representation to preserve interactivity under high n.

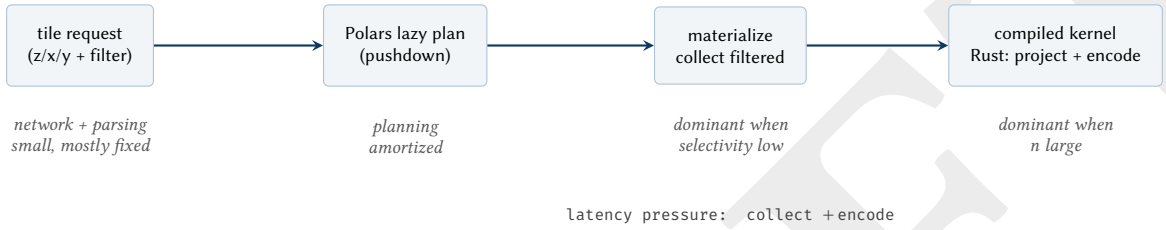


Figure 2. Latency decomposition (concept): most wall time concentrates at materialization and encoding; the system boundary places encoding in a compiled Rust kernel.

1 Problem Statement and Design Goal

Interactive map viewers frequently require serving tiles over HTTP at high request rates while selecting a small spatial slice from a large dataset. For large point clouds (10^7–10^8 events), naive JSON responses or per-point Python processing becomes the bottleneck.

The design goal of Tileserver Polars is to couple:

- **Columnar filtering** (Polars lazy expressions) for spatial and attribute predicates.
- **A Rust heavy-lifter** for coordinate transforms and encoding to MVT PBF.
- **A minimal API** (FastAPI) that is explicit, cache-friendly, and safe under untrusted query parameters.

Insight: *Tile engines must treat* tile-space projection + encoding *as a compiled kernel, not as application-layer Python.*

: Mode Switch via Density Gate

Tiles are emitted in *raw* mode when the post-filter point count is below a fixed threshold; otherwise the tile is emitted in *bins* mode where points are aggregated into an adaptive grid.

2 System Overview

The repository implements three major components:

- **API server** (FastAPI): endpoints for health, stats, TileJSON, and tile delivery.
- **Tile engine** (Python + Polars): bounding box predicate, optional user filters, and mode switch.
- **Rust extension** (PyO3): WGS84-to-tile coordinate conversion, adaptive binning, and MVT encoding.

Design Decision 1: Contracts and Boundaries

The Python layer owns: request validation, schema-aware filter compilation, and orchestration. The Rust layer owns: math and bytes. This boundary minimizes Python overhead on hot paths.

3 API Surface

The HTTP interface is intentionally small.

The tile endpoint includes response headers:

- `X-Tile-Mode`: `raw` or `bins`.
- `X-Point-Count`: post-filter point count.
- `X-Feature-Count`: emitted feature count (equals points in raw mode; bins in bins mode).

Insight: *A HEAD tile endpoint is a cheap introspection tool for clients and test harnesses. It also supports tile-cache warming strategies without bytes.*

Endpoint	Method	Purpose
`/health`	GET	Liveness check.
`/stats`	GET	Dataset row count, schema, and geographic bounds (optional filter).
`/tilejson.json`	GET	TileJSON 3.0 descriptor for client bootstrapping, including vector layer metadata.
`/tiles/{z}/{x}/{y}.mvt`	GET	Vector tile in MVT PBF format.
`/tiles/{z}/{x}/{y}.mvt`	HEAD	Metadata-only probe (mode + counts) without body.

Table 1. Primary endpoints implemented by the server.

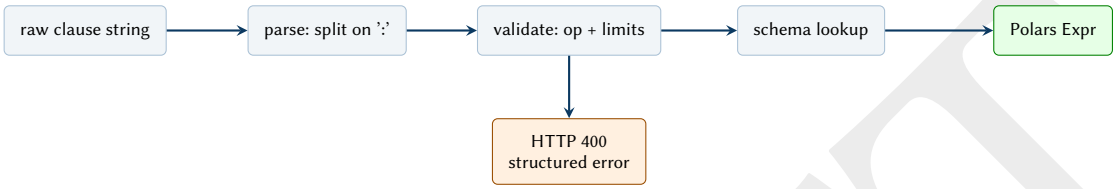


Figure 3. Filter DSL compilation pipeline: text → typed expression with explicit failure mode.

4 Filter DSL and Safety Model

The API supports repeated query parameters of the form:

`filter={field}:{op}:{value}`

Semantics are conjunction (AND) across multiple clauses.

: Allowlist Query Parameters

Each endpoint enforces an explicit allowlist of accepted query parameter keys and rejects unknown keys with an HTTP 400 error.

Supported operators include: `eq`, `in`, comparisons, string predicates, and temporal ranges. Limits are enforced (maximum clause count, maximum `in` list length, maximum string token length) to bound worst-case behavior.

5 Threat Model and Abuse Cases

The server accepts untrusted query parameters and must remain responsive under adversarial or accidental heavy requests. The threat model is focused on *resource exhaustion* rather than data exfiltration.

Primary abuse surfaces:

- **Clause explosion:** large numbers of `filter=` parameters increase parsing, schema lookups, and expression building.
- **List explosion:** `in` lists with many items can force large literal lists and large expression graphs.
- **String blowups:** long string values can create disproportionate work in string predicates.
- **Low selectivity:** broad predicates (including bbox-only) drive high n and thus stress projection/encoding.

Mitigations implemented in the repository:

- Query parameter allowlists per endpoint (reject unknown keys with HTTP 400).
- Explicit caps: `MAX_CLAUSES_PER_REQUEST`, `MAX_IN_LIST_LENGTH`, and `MAX_STRING_VALUE_LENGTH`.
- Density-gated output mode: raw points under a threshold; binned counts above it.

6 Tile Engine

Insight:
The most important safety invariant is not "filters are correct"; it is "every request has a bounded worst-case cost."

The tile engine executes:

1. Convert tile address (z,x,y) to a WGS84 bounding box.
2. Build a bounding box predicate on `longitude` and `latitude`.
3. AND with the optional user filter expression.
4. Collect the filtered frame.
5. If count $\leq \tau$, project points and encode a point layer with attributes.
6. Else, project points, bin into an adaptive grid, and encode a binned layer.

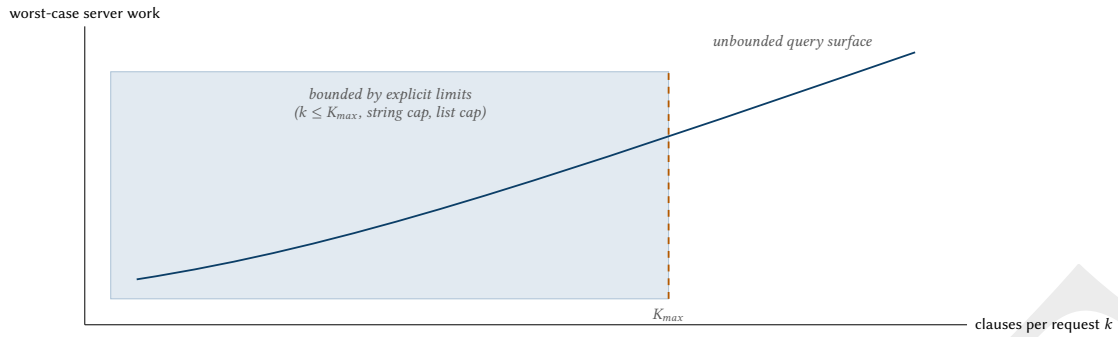


Figure 4. Performance boundary: restricting the query surface bounds worst-case work under untrusted filters.

Algorithm 1: Soft-targeted coarsening for binned tiles

Require: tile coords $\{(x_i, y_i)\}_{i=1}^{n}$, extent E, initial soft target B, max steps S
State: $t \leftarrow B$
For $s \leftarrow 0$ to S:
  $bins \leftarrow \textsc{AdaptiveGridBin}(\{(x_i, y_i)\}, E, t)$
  if $|bins| \le t$ **return** $bins$
  $t \leftarrow \max(256, \lfloor t/2 \rfloor)$
return $bins$ *(best effort after S refinements)*

Figure 5. Pseudocode listing rendered without algorithmicx to avoid PDF destination warnings.

7 Benchmarking and Performance Targets

Insight: *Collecting the filtered frame is the key* materialization point; *everything before it must remain lazy to keep optimizations intact.*

The repository includes manual microbenchmarks in the file test_engine.py intended to establish baseline performance for the Rust extension and to catch regressions.

Recommended workflow:

- Run `python test_engine.py` after changes to `src/lib.rs` or the tile pipeline.
- Use `PERF_ENFORCE=1` to turn baseline checks into assertions (CI-appropriate once thresholds are calibrated).

Design Decision 2: Raw vs Bins Semantics

Raw mode preserves point attributes (e.g., `value`, `timestamp`) when present. Bins mode emits a separate layer with per-feature `count` only.

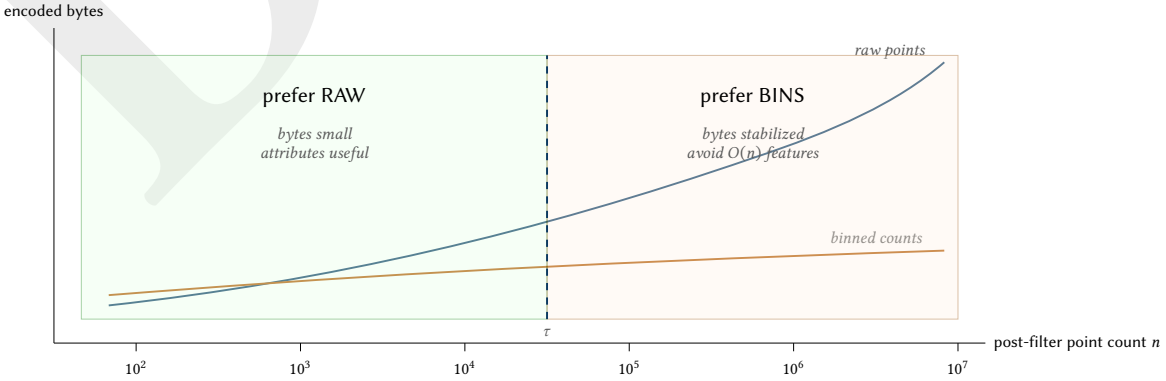
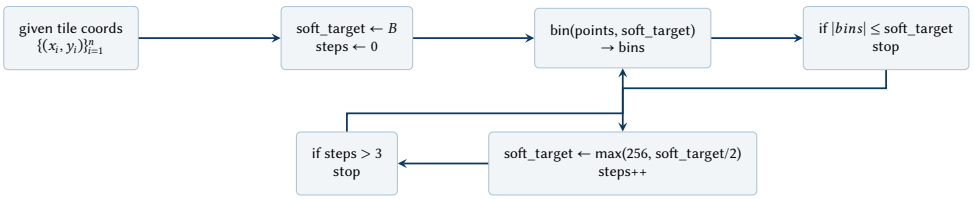


Figure 6. Bytes budget model: raw tiles grow with point count; binned tiles target a near-constant feature/byte budget.



Performance intent: enforce an approximate feature budget B while allowing a small number of corrective iterations.

```
engine.py:  coarsening loop  max_coarsen_steps=3  SOFT_BIN_TARGET=4096
```

Figure 7. Advanced algorithm sketch: soft-targeted bin coarsening loop to stabilize emitted feature count (and thus latency/bytes).

Benchmark	Intent
MVT encoding baseline	Encode 100k points with (value,timestamp) properties to estimate encoder throughput and resulting tile size.
Batch vs single projection	Compare batch coordinate transforms against repeated single-point calls to validate vectorization benefits.
Parallel vs sequential projection	Compare Rayon-parallel batch projection against sequential batch projection, validating correctness and speedup.
Concurrent deadlock check	Validate that releasing the GIL around Rayon work prevents deadlocks under concurrent calls.

Table 2. Manual benchmarks implemented in test_engine.py.

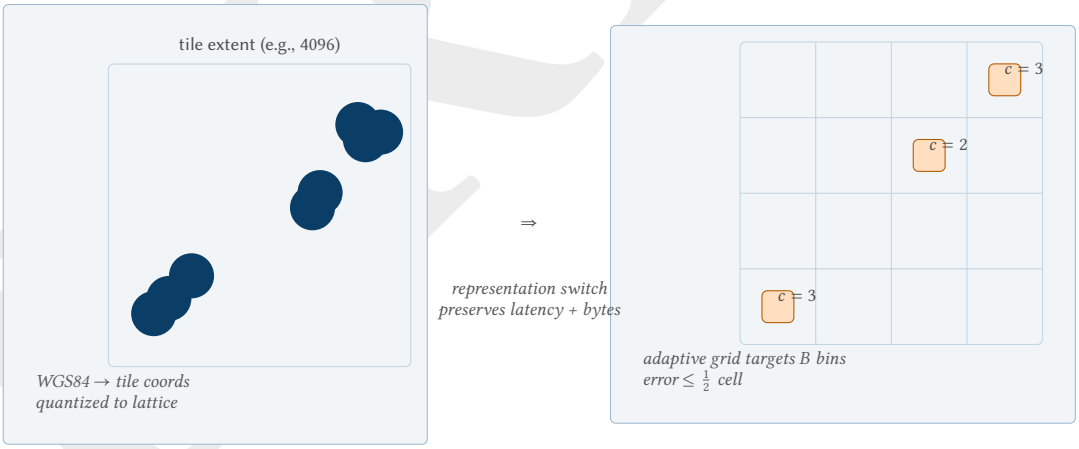


Figure 8. Algorithm geometry: point projection into tile space and the resulting adaptive grid bin representation.

Figure 9. Predicate composition: tile bounding box AND optional DSL filters are applied before collection.

Algorithm 2: MVT tag dictionary indexing (per layer)

Require: features with properties $\{(k, v)\}$, layer dicts $keys[], values[]$
State: $key\_index \leftarrow \emptyset$ (HashMap: string → index), $value\_index \leftarrow \emptyset$
For each feature f:
  $tags \leftarrow \langle\,\rangle$
  For each $(k, v) \in props(f)$:
    if $k \notin key\_index$: $key\_index[k] \leftarrow |keys|$; append k to $keys$
    if $v \notin value\_index$: $value\_index[v] \leftarrow |values|$; append v to $values$
    append $key\_index[k]$ then $value\_index[v]$ to $tags$
  set $f.tags \leftarrow tags$

Figure 10. Pseudocode listing rendered without algorithmicx to avoid PDF destination warnings.

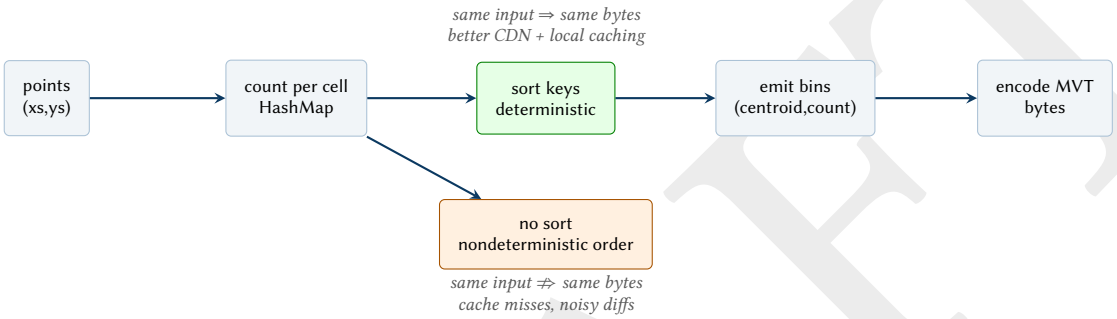


Figure 11. Performance/caching concept: determinism in bin emission improves cache hit rate and reduces downstream churn.

8 Rust Backend: Projection, Binning, and MVT Encoding

The Rust extension exposes a small surface area used by the Python engine:

- `tile_to_wgs84_bbox`: tile address → WGS84 bbox.
- `wgs84_to_tile_coords_batch`: (lon,lat) → (x,y) in tile extent.
- `bin_points_adaptive_grid_counts`: adaptive grid binning that targets a soft maximum bin count.
- `encode_tile_points_*`: MVT encoding routines.

Design Decision 3: Correctness Envelope for Projection

Projection to tile space must clamp into `[0, extent-1]` to avoid invalid MVT geometries. Batch projection applies clamping after rounding to keep output valid even when floating-point normalization produces slight out-of-range values.

: Deterministic Binning Output

Bins are emitted in deterministic order by sorting grid cell keys. Determinism reduces tile diffs and improves cache stability.

9 Viewer Integration

The repository includes a React/Vite viewer that uses Kepler.gl for interactive exploration. The API server can optionally serve the viewer SPA build output (static assets and a fallback route).

Insight: *The MVT encoder must avoid quadratic tag dictionary lookups; maintaining explicit key/value indices keeps feature emission linear.*

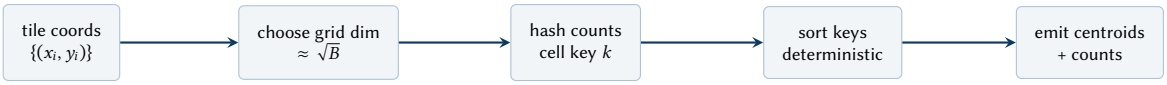


Figure 12. Adaptive grid binning: choose a grid dimension to target a soft bin budget B, count points per cell, sort keys, emit centroids.

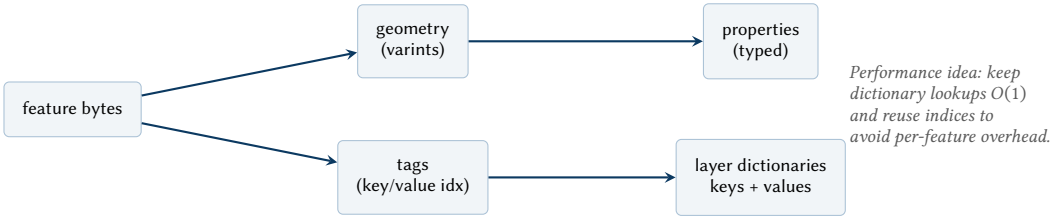
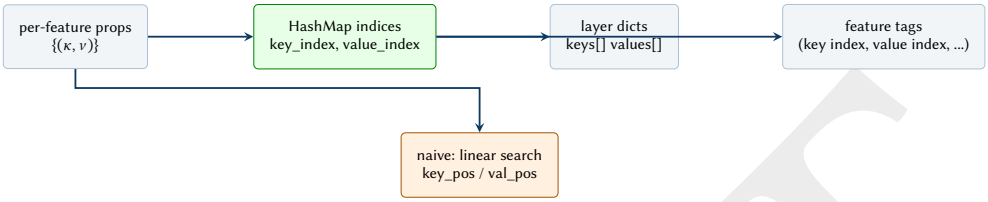


Figure 13. MVT bytes decomposition: geometry and tags; tag indices depend on layer-level key/value dictionaries.



Performance claim: dictionary indexing makes tag emission $O(|props|)$ per feature, avoiding the quadratic blow-up from repeated linear searches.

`lib.rs: encode_tile_points_impl key_index/value_index HashMaps`

Figure 14. Advanced encoder idea: explicit key/value indexing for MVT tags to keep feature emission linear in properties.

> **Design Decision 4: Single-Origin Deployment**
>
> Serving the viewer from the same origin as the API simplifies deployment and reduces CORS complexity for non-development environments.

10 Deployment Profiles

The server supports two common deployment profiles.

> **Design Decision 5: Development Profile**
>
> Run the React/Vite viewer separately (e.g., `localhost:5173`) and enable CORS to permit the browser to fetch tiles from the API (e.g., `localhost:8000`).

> **Design Decision 6: Single-Origin Production Profile**
>
> Build the viewer and serve its static assets from the API process. This avoids CORS and simplifies CDN configuration.

Key environment variables:

- `DATA_PATH`: Parquet dataset path loaded at startup.
- `HOST`, `PORT`: bind address and port.

11 Limitations and Future Work

The current system is optimized for point datasets with `longitude`/`latitude` columns.

Potential extensions include:

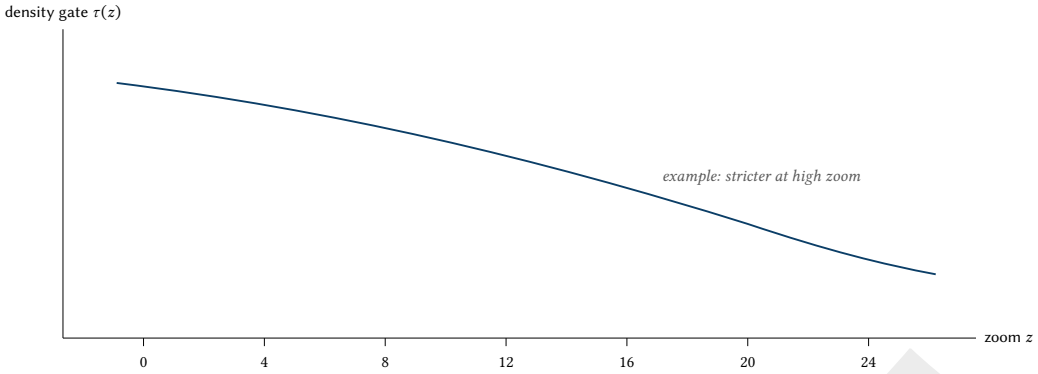
- Server-side caching (tile cache and/or result cache keyed by filter clauses).
- A zoom-aware density gate $\tau(z)$ and bin budget $B(z)$ tuned for stable p95 latency.
- Multi-layer outputs (e.g., multiple point types or multiple binning strategies).
- Typed schema export in TileJSON (field types inferred from Polars dtypes).
- End-to-end benchmarking harness that records p50/p95 tile latency and output sizes.

12 Cacheability and Canonicalization

Tile responses are cache-friendly when the request URL is a canonical key for the underlying predicate.

Current behavior:

- Tile URLs include filter clauses verbatim as repeated `filter=` query parameters.



Performance note: a fixed τ is simple, but a calibrated $\tau(z)$ can better stabilize p95 latency while preserving attributes where it matters.

Figure 15. Future work concept: zoom-aware density gating for more stable latency across scales.

- Clause order is not normalized; semantically equivalent filters in different orders produce different URLs.

Recommended canonicalization (future work):

- Sort clauses lexicographically by (field,op,value) before constructing `tiles[]` in TileJSON.
- Normalize whitespace and encoding for stable cache keys.

Insight:
Caching only works as well as the canonical form of the request. Determinism in both server output and URL structure is the performance multiplier.

List of Figures

List of Tables

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial technical report extracted from repository implementation

Todo list